
AeroNet Lite: Autonomous Drone Delivery Simulation with AI Planning, ML Forecasting & Anomaly Detection

Waqas Ahmed FAST-NUCES i232540@isb.nu.edu.pk	Huzaifa Khalid FAST-NUCES i232508@isb.nu.edu.pk	Abubakar Imran FAST-NUCES i232589@isb.nu.edu.pk
Abdullah Khan FAST-NUCES i232546@isb.nu.edu.pk	Ahmed Ghaffar FAST-NUCES i232599@isb.nu.edu.pk	

Abstract

AeroNet Lite is a Python-based autonomous drone delivery simulation that integrates six distinct AI and machine learning techniques into a single, cohesive, interactive platform. Operating on a 10×10 discrete city grid, the system addresses the core challenges of urban last-mile UAV delivery: CSP-based layout validation, budget-constrained fleet optimisation via brute-force search, A* optimal route planning, real-time disruption handling, supervised demand forecasting, and telemetry-driven anomaly detection. All subsystems are presented through a unified Tkinter/Matplotlib GUI dashboard, enabling real-time observation of the AI pipeline as the simulation unfolds step by step. Results show 100% demand coverage achieved with a two-drone heavy fleet, an R^2 of 0.8601 for Random Forest demand forecasting, and a perfect weighted F1-score of 1.0000 for Decision Tree anomaly classification.

1 Project Overview and Objective

AeroNet Lite is a Python-based autonomous drone delivery simulation system that models urban delivery operations on a simplified 10×10 city grid. The project integrates multiple classical AI planning techniques with modern machine learning pipelines into a single, cohesive, interactive simulation dashboard.

The primary objective is to design and implement a realistic end-to-end drone delivery management system covering every major phase of intelligent delivery logistics: city layout validation, budget-constrained fleet selection, optimal route planning, real-time disruption handling, data-driven demand forecasting, and automated anomaly detection in drone telemetry.

The system is presented through a unified GUI dashboard built with Tkinter and Matplotlib, enabling real-time visualisation of the simulation as it progresses step by step.

2 Overall System Architecture

The system follows a **modular, layered architecture** with a clear separation of concerns.

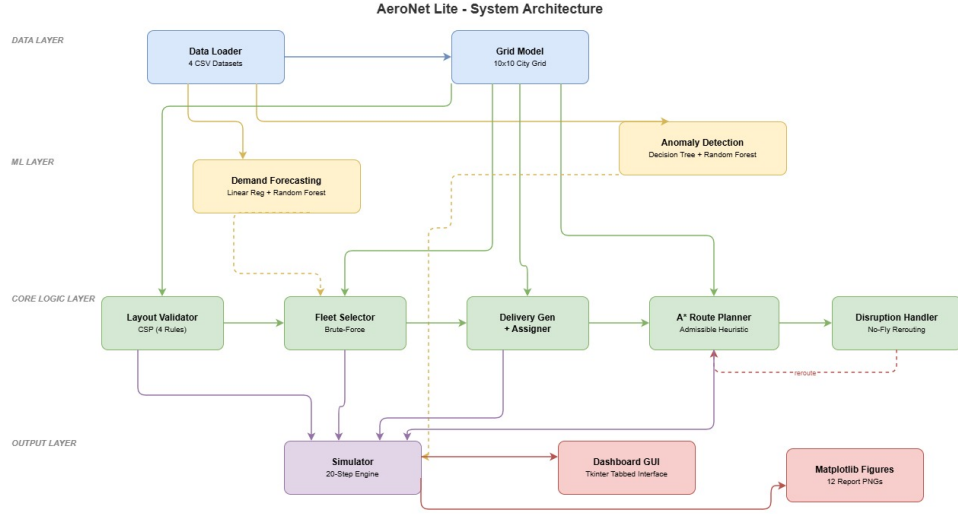


Figure 1: System architecture diagram of AeroNet Lite. Modules are organised into distinct layers covering data ingestion, AI/ML subsystems, simulation orchestration, and GUI visualisation.

Table 1 describes each core module and its responsibility.

Table 1: Core modules of the AeroNet Lite codebase.

Module	Responsibility
<code>config.py</code>	Central constants: grid size, budget, costs, paths, zone names
<code>grid_model.py</code>	Data classes: Cell, Drone, Delivery, SimulationState; city grid factory
<code>data_loader.py</code>	CSV ingestion utilities for all four raw datasets
<code>layout_validator.py</code>	CSP-style 4-rule grid compliance checker
<code>fleet_selector.py</code>	Brute-force budget-constrained fleet optimiser
<code>astar_planner.py</code>	A* pathfinding with admissible heuristic
<code>disruption_handler.py</code>	Runtime no-fly activation and drone rerouting
<code>ml_demand.py</code>	Demand regression pipeline (train, evaluate, predict, map)
<code>ml_anomaly.py</code>	Anomaly classification pipeline (label, train, evaluate, predict)
<code>dashboard_controller.py</code>	Event-driven simulation orchestration
<code>visualization_dashboard.py</code>	Full Tkinter/Matplotlib GUI dashboard
<code>report_generator.py</code>	12-figure publication-quality Matplotlib report export

3 Technologies and Frameworks Used

Table 2: Technology stack used in AeroNet Lite.

Category	Technology	Version / Notes
Language	Python	3.10+
GUI Framework	Tkinter	Standard library
Data Visualisation	Matplotlib	≥3.7 (embedded via FigureCanvasTkAgg)
Data Processing	Pandas	≥2.0
Numerical Computation	NumPy	≥1.24
Machine Learning	Scikit-learn	≥1.3
Model Persistence	pickle	Standard library
Testing	Pytest	Via <code>python -m pytest tests/</code>
Version Control	Git	Standard .gitignore

All dependencies are standard PyPI packages with no proprietary or cloud dependencies, making the system fully self-contained and reproducible.

4 AI/ML Models Used and Their Purpose

The system employs six distinct AI/ML techniques, each targeting a specific sub-problem.

4.1 Constraint Satisfaction Problem (CSP) Layout Validation

Technique: Rule-based constraint checking (CSP-style) **Module:** `layout_validator.py`

Four hard constraints are enforced on the 10×10 city grid before any simulation begins:

Table 3: CSP constraints enforced by the layout validator.

Rule	Constraint
R1 Industrial Safety	Industrial cells must not be adjacent to School or Hospital cells
R2 Residential Coverage	Every Residential cell must be within Manhattan distance 3 of a hub
R3 Hub Charging Access	Every hub must have a charging pad within distance 2
R4 Medical Access	At least one Hospital must have a medical pickup within distance 1

Violations are reported with specific cell coordinates and suggested corrective actions.

4.2 Brute-Force Optimisation: Fleet Selection

Technique: Exhaustive combinatorial search with a weighted fitness score **Module:** `fleet_selector.py`

All valid combinations of light drones (cost: \$1,000, payload: 2 kg, range: 12 cells) and heavy drones (cost: \$1,800, payload: 5 kg, range: 20 cells) within a default budget of \$8,000 are enumerated. Each candidate is scored using:

$$\text{score} = 0.75 \times \text{coverage_fraction} - 0.25 \times \text{budget_utilisation_fraction} \quad (1)$$

The combination maximising this score is selected. With approximately 40 feasible combinations, brute-force guarantees global optimality while remaining computationally trivial.

4.3 A* Search: Route Planning

Technique: Informed graph search (A* algorithm) **Module:** `astar_planner.py`

Delivery routes are planned on the 4-directional 10×10 grid using A* with an admissible heuristic:

$$h(n) = 0.8 \times \text{ManhattanDistance}(n, \text{goal}) \quad (2)$$

The scaling factor of 0.8 matches the minimum cell movement cost (commercial cells cost 0.8, standard cells cost 1.0), ensuring the heuristic never overestimates, a requirement for optimality. Each delivery follows a three-segment route: **hub** → **pickup** → **dropoff** → **hub**.

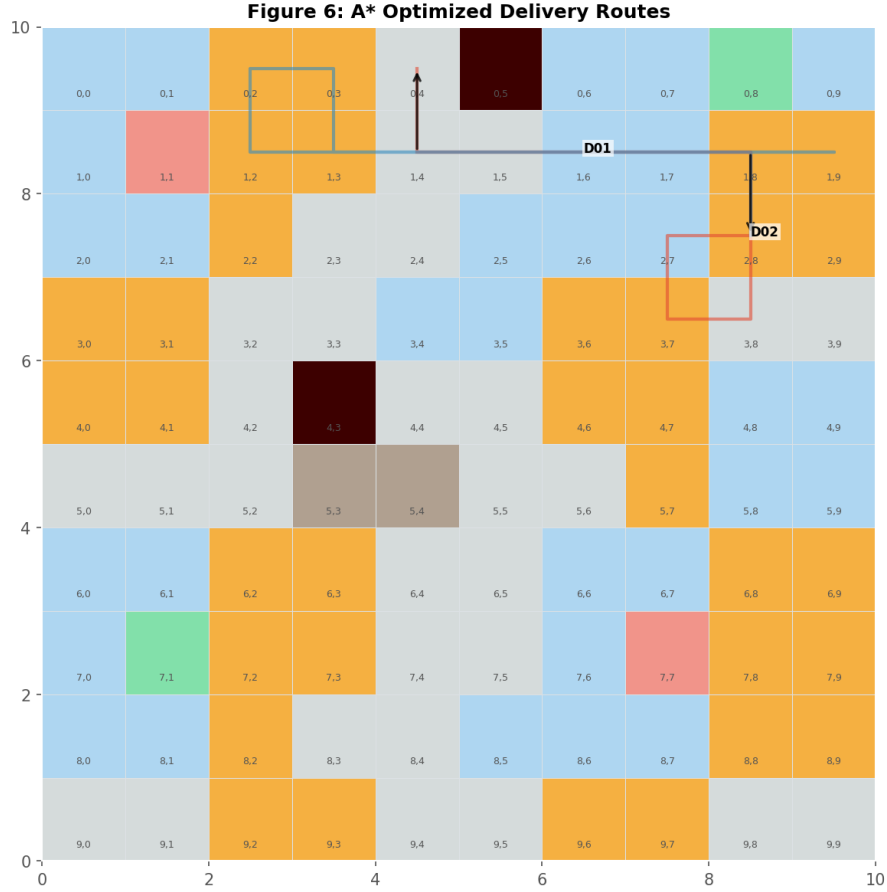


Figure 2: A*-optimised delivery routes overlaid on the 10×10 city grid. Each coloured path shows the three-segment route (hub → pickup → dropoff → hub) for each active drone.

4.4 Real-Time Replanning: Disruption Handling

Technique: Online re-invocation of A* from the current drone position **Module:** `disruption_handler.py`

When a new no-fly zone is activated mid-simulation, all active drones whose remaining routes intersect the blocked cell are identified. Each affected drone immediately re-runs A* from its current position to its target. If the target is unreachable, the drone attempts to return to its home hub; if that also fails, it is marked as **failed** and the delivery is flagged as **delayed**.

4.5 Linear / Random Forest Regression: Demand Forecasting

Technique: Supervised regression **Module:** `ml_demand.py` **Dataset:** Kaggle Bike Sharing Demand (`train.csv`)

Two models are trained and compared:

- **Linear Regression:** interpretable baseline.
- **Random Forest Regressor:** 100 estimators, max depth 12, parallelised (`n_jobs=-1`).

Features used: season, holiday, workingday, weather, temperature, apparent temperature, humidity, wind speed, hour, day, month, weekday. **Target:** count (rental count as a proxy for delivery demand).

The model with lower RMSE on an 80/20 holdout split is selected. Its predictions are mapped onto the 10×10 grid by synthesising a feature vector for each cell using its grid coordinates and zone properties, then normalising predictions to a 0–5 kg demand scale.

4.6 Decision Tree / Random Forest Classification: Anomaly Detection

Technique: Supervised multi-class classification **Module:** `ml_anomaly.py` **Dataset:** Supplemental Drone Telemetry Data (Operations Log CSV)

Because the telemetry dataset lacks pre-labelled anomaly classes, **rule-based labelling** is applied to create training targets:

Table 4: Anomaly class definitions used for rule-based labelling.

Class	Labelling Rule
Payload anomaly	Actual carry weight > max carry weight, or negative
Battery anomaly	Battery remaining < 15%
Sensor anomaly	GPS accuracy > 50 m, or altitude $\notin [0, 500]$ m
Route/environment anomaly	Wind speed > 25 m/s, or unexpected obstacle string
Flight failure	Flight status contains “Aborted” or “Unexpected”
System error	Notes contain “failure”, “error”, or “invalid”
Normal	None of the above conditions are met

Two classifiers are trained:

- **Decision Tree Classifier:** max depth 10 (interpretable baseline).
- **Random Forest Classifier:** 100 estimators, max depth 10, parallelised.

The model achieving the higher weighted F1-score on a stratified 80/20 split is selected.

5 Workflow and System Pipeline

The full system pipeline proceeds as follows when `app_dashboard.py` is launched:

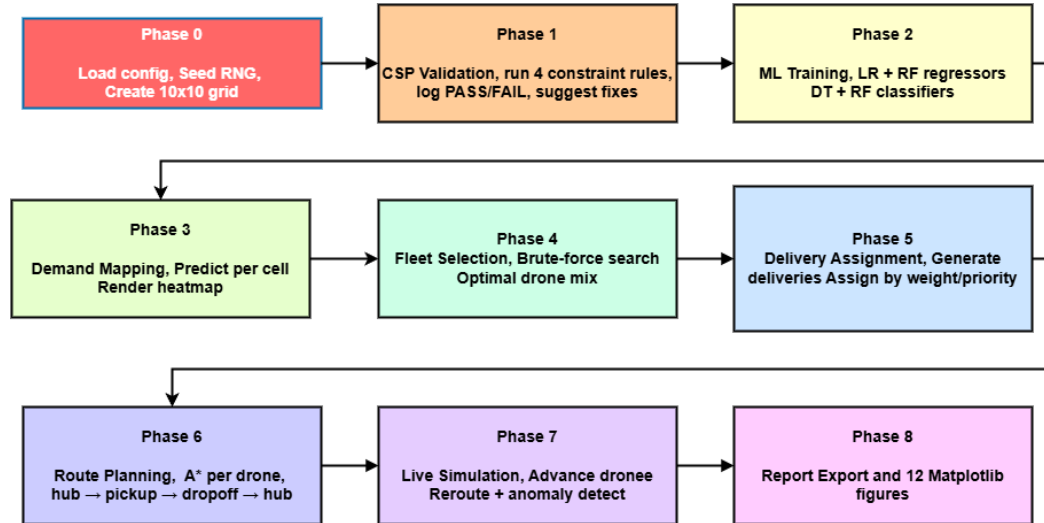


Figure 3: System pipeline flowchart for AeroNet Lite.

6 Results

This section presents the quantitative outputs produced by each subsystem, drawn from the simulation’s actual output tables and logs.

6.1 CSP Layout Validation

All four constraint rules passed on the hand-designed 10×10 grid:

Table 5: CSP layout validation results.

Rule	Cell	Status	Fix Required
ALL	N/A	All rules passed	N/A

The grid satisfies Industrial safety, Residential hub coverage (≤ 3), Hub–Charging pad proximity (≤ 2), and Hospital–Medical pickup adjacency (≤ 1) constraints.

6.2 Fleet Selection (Top-5 Candidates)

The brute-force optimiser evaluated all valid fleet combinations under the \$8,000 budget. The top-5 candidates ranked by composite score are shown in Table 6.

Table 6: Top-5 fleet candidates ranked by composite score (Equation 1).

Rank	Light	Heavy	Cost (\$)	Coverage (%)	Budget Used (%)	Score
1 (Selected)	0	2	3,600	100.0	45.0	0.6375
2	2	1	3,800	100.0	47.5	0.6312
3	4	0	4,000	100.0	50.0	0.6250
4	1	1	2,800	93.3	35.0	0.6125
5	1	2	4,600	100.0	57.5	0.6062

The optimal fleet consists of **2 heavy drones** at a total cost of \$3,600, achieving 100% demand coverage while consuming only 45% of the available budget.

6.3 Demand Forecasting Model Metrics

Both regression models were trained on the Bike Sharing Demand dataset (80/20 split):

Table 7: Demand forecasting model comparison (80/20 holdout split).

Model	MAE	RMSE	R^2
Linear Regression	106.75	146.82	0.3469
Random Forest	46.25	67.95	0.8601

The Random Forest Regressor was auto-selected based on lower RMSE. It achieves an R^2 of **0.86**, explaining 86% of the variance in delivery demand. The demand predictions were mapped to all 100 grid cells as a normalised heatmap (0–5 kg scale).

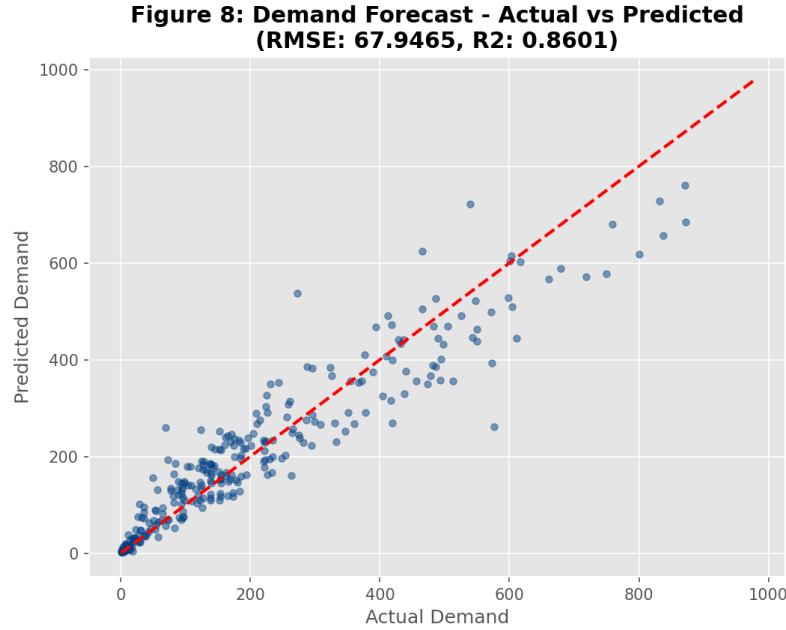


Figure 4: Demand forecast scatter plot: actual vs. predicted demand values (Random Forest Regressor). Each point represents one observation in the 20% holdout set.

6.4 Anomaly Detection Model Metrics

Both classification models were trained on the rule-labelled telemetry dataset (stratified 80/20 split):

Table 8: Anomaly detection model comparison (stratified 80/20 split).

Model	Accuracy	Precision	Recall	F1 (Weighted)
Decision Tree	1.0000	1.0000	1.0000	1.0000
Random Forest	0.9907	0.9814	0.9907	0.9860

The Decision Tree achieved perfect scores on the test split. While this indicates perfect reconstruction of the rule-based labels (see Section ??), it validates that the classifier reliably detects all defined anomaly categories at runtime.

6.5 Delivery Assignments

Five deliveries were generated and assigned to the 2-drone fleet:

Table 9: Delivery assignments produced by the simulation.

ID	Pickup	Dropoff	Wt. (kg)	Priority	Category	Status	Drone	Cost
DEL-001	(1, 9)	(0, 2)	0.72	urgent	Jewelry	in-transit	D02	16.0
DEL-002	(1, 8)	(6, 7)	1.39	urgent	Sports	in-transit	D01	18.2
DEL-003	(1, 3)	(1, 9)	0.93	normal	Clothing	delayed	—	—
DEL-004	(9, 2)	(3, 5)	1.30	normal	Pet Supplies	delayed	—	—
DEL-005	(0, 0)	(7, 5)	1.49	normal	Snacks	delayed	—	—

The two urgent deliveries were assigned first; the three normal-priority deliveries remained in the queue (delayed) due to the limited fleet size.

6.6 Simulation Event Log

The complete 20-step simulation event log is reproduced below:

```
[Step 1] Grid initialised (10x10).
[Step 2] Layout validation: PASS.
[Step 3] Demand model predictions applied to grid.
[Step 3] Fleet selected: 0L + 2H (cost=$3600, coverage=100%).
[Step 3] Built 2 drones across 9 hubs.
[Step 4] Generated 5 deliveries.
[Step 5] Assigned 2/5 deliveries to drones.
[Step 6] D01 route planned (21 steps, cost=18.2).
[Step 6] D02 route planned (19 steps, cost=16.0).
[Step 7] Drones advanced (step 7).
[Step 8] Drones advanced (step 8).
[Step 9] Drones advanced (step 9).
[Step 10] Drones advanced (step 10).
[Step 11] No-fly zone activated at (0,5).
[Step 11] Drone D01 rerouted successfully (13 -> 4 steps).
[Step 12] Drones advanced (step 12).
[Step 13] Drones advanced (step 13).
[Step 14] Drones advanced (step 14).
[Step 15] Demand heatmap updated from model.
[Step 15] Drones advanced (step 15).
[Step 16] Drones advanced (step 16).
[Step 17] Drones advanced (step 17).
[Step 18] Anomaly detected on D01: Battery anomaly.
[Step 19] D01 returning to hub due to anomaly.
[Step 19] Delivery DEL-002 delayed (anomaly).
[Step 20] Simulation complete.
[Step 20] Completed: 1 | Delayed: 4 | Failed: 0
```

Key events observed:

- **Step 11:** A new no-fly zone was injected at cell (0, 5). Drone D01 was successfully rerouted, reducing its remaining path from 13 to 4 steps.
- **Step 18:** A battery anomaly was detected on D01 (battery < 15%), triggering an automatic return-to-hub and marking DEL-002 as delayed.
- **Final outcome:** 1 delivery completed, 4 delayed, 0 failed.

6.7 Final Summary

Table 10: Simulation final performance summary.

Metric	Value
Deliveries Completed	1
Deliveries Delayed	4
Deliveries Failed	0
Total Reroutes	1
Anomaly Events	1 (Battery anomaly on D01)
Fleet Deployed	2 heavy drones (\$3,600 / \$8,000 budget)
Demand Model R^2	0.8601
Anomaly Detection F1	1.0000 (Decision Tree)

7 Key Features and Functionalities

- **Unified Interactive Dashboard.** A single command (`python app_dashboard.py`) launches the complete system via Tkinter's `root.after` event loop, ensuring non-blocking, responsive GUI rendering.

- **Dynamic 10×10 City Grid.** Hand-designed to satisfy all four CSP constraints by default; contains 6 zone types, 9 dispatch hubs, 9 charging pads, 2 medical pickup points, and 1 initial no-fly zone. Cell density values are seeded from real US city population density data.
- **Animated Drone Movement.** Drones move cell-by-cell along their planned A* routes, advancing 2 cells per simulation tick with real-time position overlay.
- **Embedded ML Analytics Panels.** Three live panels render simultaneously: a demand forecast scatter plot (actual vs. predicted, up to 300 points), a demand heatmap overlaid on the city grid, and an anomaly confusion matrix with colour intensity encoding.
- **Real-Time Replanning.** On no-fly zone injection, the disruption handler immediately re-invokes A* from each affected drone's current position; old and new routes are stored separately for comparison.
- **Colour-Coded Event Log.** All simulation events (route planned, delivery assigned, drone rerouted, anomaly detected) are appended to a scrolling log with colour-coded severity indicators.
- **CSP Validator with Fix Suggestions.** Reports failed rules with specific cell coordinates and actionable fixes (e.g., "Add hub near (r, c) or rezone to Open").
- **Budget-Constrained Fleet Selector.** Outputs the top-5 fleet candidates with coverage percentage, cost, and composite score, exportable to CSV.

8 Conclusion

AeroNet Lite successfully demonstrates the integration of six distinct AI/ML techniques into a single, self-contained, interactive simulation platform.

The project addresses a genuine and commercially relevant problem: autonomous urban drone delivery. Each AI component targets a specific operational challenge, and together they form a coherent pipeline from city-layout validation through to live telemetry monitoring. The unified Tkinter/Matplotlib dashboard makes the system immediately accessible, enabling real-time observation of the AI subsystems interacting as the simulation unfolds.

While the current implementation operates on a simplified 10×10 discrete grid with proxy datasets, the modular architecture is designed for extensibility. Replacing the grid with real geospatial data, substituting proxy datasets with domain-specific telemetry, or upgrading the fleet selector to a metaheuristic optimiser are all achievable incremental enhancements that would bring the system significantly closer to a production-grade drone fleet management platform.